

Curso de Go

Aula 16 - Servidor web básico com net/http

Professor : Antônio Marcelo

NÚCLEA
ACADEMY

 **C/B** [código_] **BRAZUCA** [3]





Nessa Aula nos Vamos Ver

- **Nesta aula vamos aprender como criar um servidor web utilizando apenas a biblioteca padrão do Go, sem frameworks externos. O pacote net/http é extremamente poderoso e utilizado em aplicações reais de alta performance.**
- **Parte Prática**



Introdução ao net/http em Go

- O Go possui um modelo simples para construção de APIs:

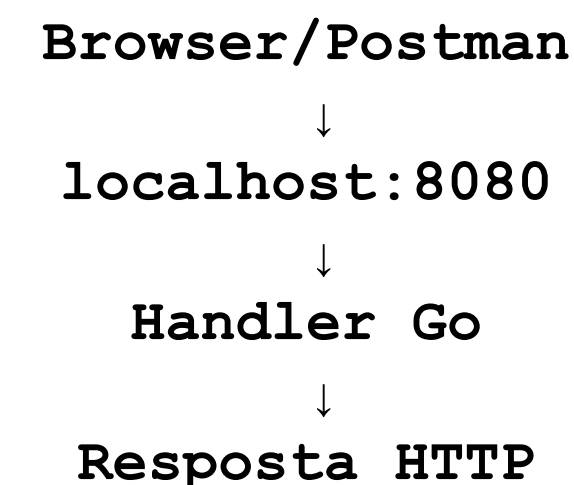
- Um servidor escuta conexões HTTP
- Rotas recebem requisições
- Handlers processam as requisições
- Responses retornam dados ao cliente

- Vantagens do net/http:

- Faz parte da biblioteca padrão
- Alto desempenho
- Simples de aprender
- Muito utilizado em microsserviços
- Excelente concorrência nativa

- Fluxo básico: Cliente → Servidor → Handler → Resposta

Exemplo conceitual:





Estrutura mínima de um servidor HTTP

- Um servidor HTTP básico em Go pode ser criado com poucas linhas.

- Exemplo:

```
package main

import (
    "fmt"
    "net/http"
)

func main() {

    http.HandleFunc("/", homeHandler)

    fmt.Println("Servidor rodando em
http://localhost:8080")

    http.ListenAndServe(":8080", nil)
}

func homeHandler(w http.ResponseWriter, r
*http.Request) {
    fmt.Fprintf(w, "Olá Mundo!")
}
```

Explicação:

- `HandleFunc()` registra uma rota
- `/` representa a rota raiz
- `homeHandler` é a função executada
- `ListenAndServe()` inicia o servidor
- `:8080` indica a porta utilizada

Teste:

Abra o navegador e digite
`http://localhost:8080`

Resultado:

Olá Mundo!



Entendendo o Handler

- O Handler é a função responsável por processar requisições HTTP.

```
func handler(w http.ResponseWriter, r *http.Request)
```

- Parâmetros:
- **w** → envia resposta ao cliente
- **r** → contém dados da requisição

Exemplo:

```
func helloHandler(w http.ResponseWriter, r
*http.Request) {

    fmt.Println("Nova requisição recebida")

    w.Write([]byte("Bem-vindo ao servidor"))
}
```




Entendendo o Handler

- **O objeto Request contém:**
 - **Método HTTP**
 - **Headers**
 - **Query params**
 - **Body**
 - **Cookies**
 - **URL**

Exemplo:

```
fmt.Println(r.Method)
fmt.Println(r.URL.Path)
fmt.Println(r.RemoteAddr)
```

```
Saída
GET
/hello
127.0.0.1:52144
```




Criando múltiplas rotas

- Um servidor normalmente possui várias rotas. Exemplo:

```
package main

import (
    "fmt"
    "net/http"
)

func main() {

    http.HandleFunc("/", home)
    http.HandleFunc("/sobre", sobre)
    http.HandleFunc("/contato", contato)

    fmt.Println("Servidor iniciado")

    http.ListenAndServe(":8080", nil)
}

func home(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintf(w, "Página inicial")
}

func sobre(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintf(w, "Página sobre")
}

func contato(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintf(w, "Página de contato")
}
```



Métodos HTTP

- Os métodos HTTP representam ações.
- Principais métodos:
 - **GET** → buscar dados
 - **POST** → criar dados
 - **PUT** → atualizar dados
 - **DELETE** → remover dados
- Resultados:
 - **GET** -> Listando usuários
 - **POST** -> Criando usuário
 - **PUT** -> 405 Method Not Allowed
- Boa prática: Sempre validar o método HTTP.

Exemplo verificando método:

```
func userHandler(w http.ResponseWriter, r
*http.Request) {

    if r.Method == "GET" {
        w.Write([]byte("Listando usuários"))
        return
    }

    if r.Method == "POST" {
        w.Write([]byte("Criando usuário"))
        return
    }

    http.Error(w, "Método não permitido",
http.StatusMethodNotAllowed)
}
```




Trabalhando com Query Parameters

- Query parameters são dados enviados pela URL.

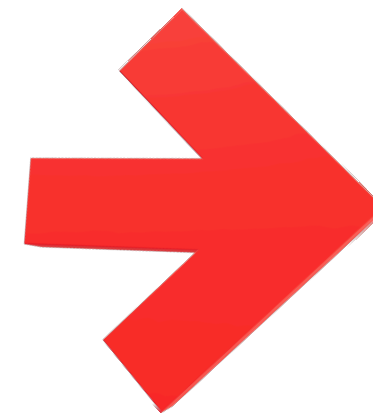
```
http://localhost:8080/produto?id=10
```

- Capturando parâmetros:

```
func produtoHandler(w http.ResponseWriter, r *http.Request) {  
    id := r.URL.Query().Get("id")  
  
    fmt.Fprintf(w, "Produto ID: %s", id)  
}
```

- Múltiplos parâmetros:

```
query := r.URL.Query()  
  
id := query.Get("id")  
nome := query.Get("nome")
```



- Muito utilizado em:

- Paginação
- Filtros
- Pesquisas
- Ordenação



APIs modernas e integração entre sistemas

- **As APIs são hoje a principal forma de comunicação entre sistemas modernos. Elas permitem que aplicações diferentes troquem informações de maneira padronizada, segura e escalável, independentemente da linguagem utilizada no backend ou frontend.**
- **APIs possibilitam integração entre aplicações web, aplicativos mobile, sistemas corporativos, IoT, microsserviços e plataformas externas.**
- **APIs REST bem projetadas facilitam manutenção, escalabilidade, documentação e reutilização de serviços em diferentes projetos.**



Estrutura mínima de uma API

- Uma API organizada costuma separar:

- main
- rotas
- handlers
- models
- services

- Exemplo estrutural:

```
project/
|
|— main.go
|— handlers/
|   |— user.go
|— models/
|   |— user.go
|— services/
|   |— user_service.go
```

exemplo simples:

```
package handlers
```

```
import (
    "encoding/json"
    "net/http"
)
```

```
func GetUsers(w http.ResponseWriter, r
*http.Request) {

    users := []string{"Ana", "Carlos"}

    w.Header().Set("Content-Type",
"application/json")

    json.NewEncoder(w).Encode(users)
}
```



Tratamento de erros HTTP

- Devemos retornar erros corretamente.
- Exemplo:

```
func exemploHandler(w
http.ResponseWriter, r *http.Request) {

    id := r.URL.Query().Get("id")

    if id == "" {
        http.Error(w, "ID obrigatório",
http.StatusBadRequest)
        return
    }

    w.Write([]byte("ID recebido"))
}
```

Resposta:
400 Bad Request

- Códigos importantes:
 - 200 → OK
 - 201 → Created
 - 400 → Bad Request
 - 404 → Not Found
 - 500 → Internal Server Error



Testando a API

- Temos como testar APIs usando Ferramentas comuns como:

- Browser
- Postman
- Insomnia
- curl

Exemplo curl:

```
curl http://localhost:8080
```

Exemplo GET:

```
curl http://localhost:8080/users
```

Exemplo POST

```
curl -X POST http://localhost:8080/users
```

Com JSON:

```
curl -X POST \  
-H "Content-Type: application/json" \  
-d '{"name": "Antonio"}' \  
http://localhost:8080/users
```

- Testes são fundamentais para:

- Validar rotas
- Verificar erros
- Garantir respostas corretas



Projeto completo de servidor básico

```
package main

import (
    "encoding/json"
    "fmt"
    "net/http"
)

type Product struct {
    ID int `json:"id"`
    Name string `json:"name"`
}

func main() {

    http.HandleFunc("/", homeHandler)
    http.HandleFunc("/products", productsHandler)

    fmt.Println("Servidor rodando em :8080")

    http.ListenAndServe(":8080", nil)
}

func homeHandler(w http.ResponseWriter, r *http.Request) {

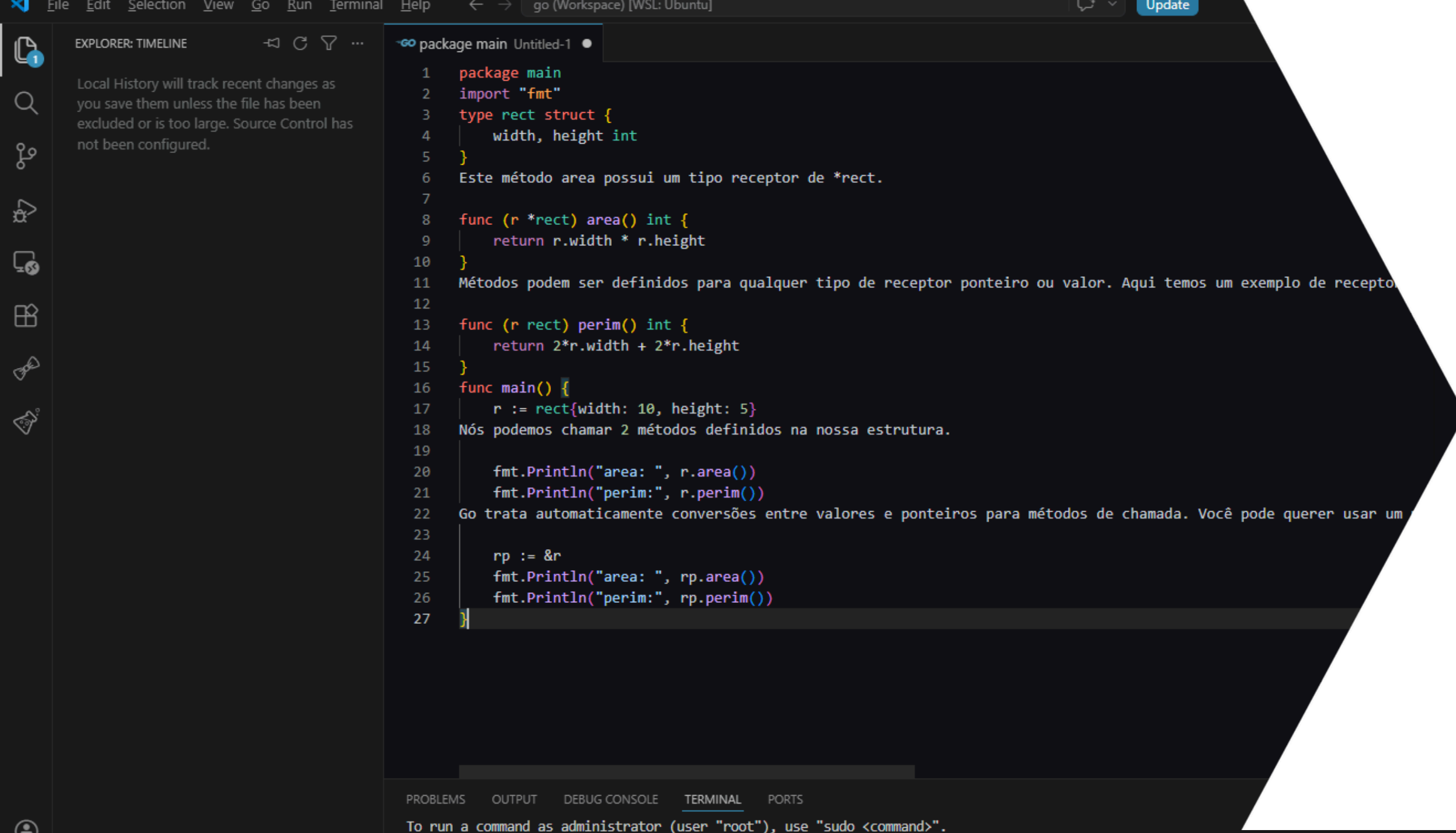
    w.Write([]byte("API Online"))
}

func productsHandler(w http.ResponseWriter, r *http.Request) {

    products := []Product{
        {ID: 1, Name: "Notebook"},
        {ID: 2, Name: "Mouse"},
    }

    w.Header().Set("Content-Type", "application/json")

    json.NewEncoder(w).Encode(products)
}
```



The screenshot shows a Go IDE with a file named 'package main' in 'Untitled-1'. The code defines a 'rect' struct with 'width' and 'height' fields. It includes two methods: 'area()' which returns the area, and 'perim()' which returns the perimeter. The 'main' function creates a 'rect' with width 10 and height 5, prints its area and perimeter, and then demonstrates calling these methods on a pointer to the 'rect'.

```
1 package main
2 import "fmt"
3 type rect struct {
4     width, height int
5 }
6 Este método area possui um tipo receptor de *rect.
7
8 func (r *rect) area() int {
9     return r.width * r.height
10 }
11 Métodos podem ser definidos para qualquer tipo de receptor ponteiro ou valor. Aqui temos um exemplo de recepto
12
13 func (r rect) perim() int {
14     return 2*r.width + 2*r.height
15 }
16 func main() {
17     r := rect{width: 10, height: 5}
18     Nós podemos chamar 2 métodos definidos na nossa estrutura.
19
20     fmt.Println("area: ", r.area())
21     fmt.Println("perim:", r.perim())
22     Go trata automaticamente conversões entre valores e ponteiros para métodos de chamada. Você pode querer usar um
23
24     rp := &r
25     fmt.Println("area: ", rp.area())
26     fmt.Println("perim:", rp.perim())
27 }
```

NÚCLEA
ACADEMY

Parte Prática

Servidor web básico com net/http